알고리즘1 (2026-1)

4. 균형 이진 탐색 트리 (AVL 트리)

국립금오공과대학교 컴퓨터공학부

컴퓨터공학전공

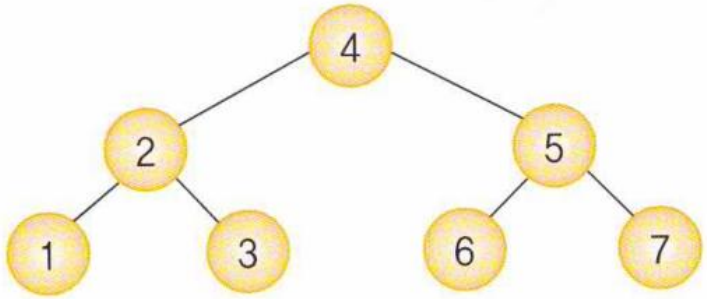
김 경 수

학습목표

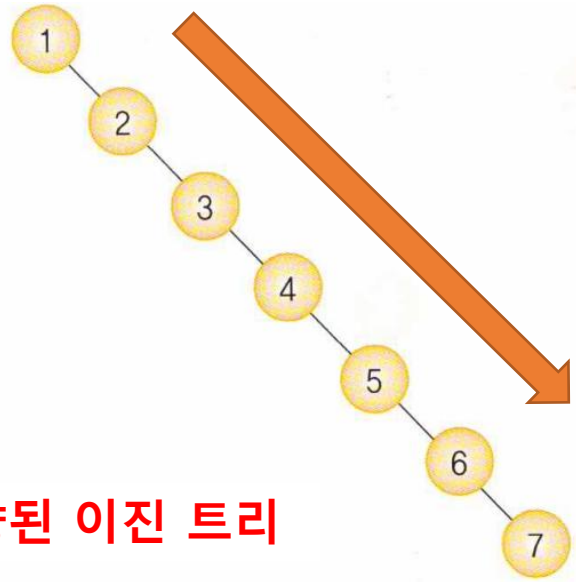
- 균형 이진 트리의 개념과 필요성을 이해하고 설명할 수 있다.
- 대표적인 균형 이진 트리인 AVL 트리의 개념을 이해하고, 서브 트리 간 높이 균형을 자동으로 조정하는 방법을 설명할 수 있다.
- 프로그래밍 언어를 이용하여 실제 AVL 트리를 구현하고, 테스트 케이스를 이용하여 구현 결과를 확인할 수 있다.

편향된 이진 트리

- 이진 탐색 트리(binary search tree)에서 데이터의 크기 순으로 순차적으로 입력하는 경우 이진 탐색 트리의 편향 문제가 발생
- 트리의 총 노드의 수가 n 일 때, 편향된 이진 트리의 높이는 최대 n.
  - ➢ 이 경우 탐색 소요시간은 최대 $O(n)$으로 선형 탐색과 다름이 없음.
  - ➢ 따라서 트리의 높이를 $\log n$에 가깝게 유지해야 이진 탐색 트리에서의 효율적인 탐색(즉, $O(\log n)$)을 기대할 수 있음.



높이가 $\log n$인 이진 트리



오른쪽으로 편향된 이진 트리

균형 이진 탐색 트리

- 트리의 높이가 항상 $\log n$ 에 가깝게 유지될 수 있도록, 서브 트리 간의 높이 균형을 자동으로 조정하는 이진 탐색 트리.
- 새로운 데이터가 삽입되거나 기존 데이터가 삭제될 때, 트리의 높이가 $\log n$ 에 가깝게 유지될 수 있도록 노드들의 위치를 자동으로 조정한다.
- 균형 이진 탐색 트리의 종류
  - AVL 트리
  - Red-Black 트리
  - Splay 트리

  …

AVL Tree 개요

- 1962년에 Adelson-Velsky와 Landis에 의해서 개발된 균형 이진 탐색 트리
- AVL 트리의 정의
  - ➢ **임의의 이진 탐색 트리 T가 <u>높이 균형</u>일 때 T를 AVL 트리라고 한다.**

★ **높이 균형(height-balance property)**

트리 $\boldsymbol{T} = \{\boldsymbol{V}, \boldsymbol{E}\}$의 모든 노드 집합이 $\boldsymbol{V}$ 이고 각 노드를 $v \in \boldsymbol{V}$라 할 때, v 의 자식 노드들의 높이 차이가 최대 ±1인 경우 $\boldsymbol{T}$는 높이 균형을 만족한다.

두 서브 트리의 높이 차이 계산하기

- **노드 A의 균형 인수**

  = 서브트리 높이 차이

  = 왼쪽 서브트리의 깊이(depth) – 오른쪽 서브트리의 깊이(depth)

  ※ 이 때 노드 개수가 한 개인 경우 depth=0.

- 노드 A의 균형 인수가 2 이상이거나, -2 이하인 경우 "노드 A에서 트리 높이의 불균형이 발생"한 것으로 간주한다.
  - ➢ 이를 해결하기 위해, 노드 A와 두 서브트리의 위치를 조정하여 높이 차이(즉, 균형 인수)가 -1에서 1 사이가 되도록 조정한다.

AVL 트리에서 균형이 깨지는 경우

- **AVL 트리에서 균형이 깨지는 경우**

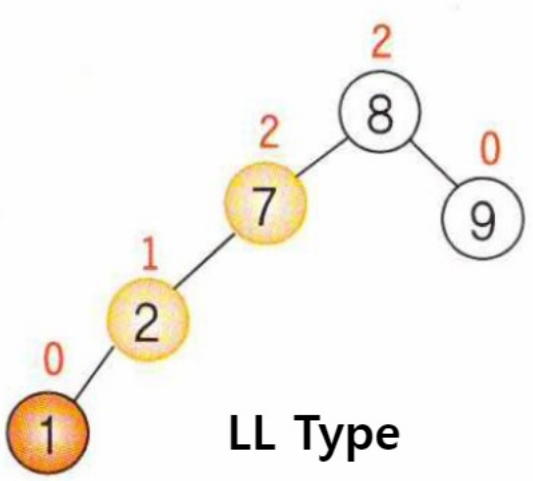
  새로 삽입된 노드를 N, N과 가장 가까우면서 균형 인수가 ±2인 조상 노드를 A라 하자.

  ① **LL Type**: N이 A의 왼쪽 서브 트리의 왼쪽 서브 트리에 삽입되는 경우

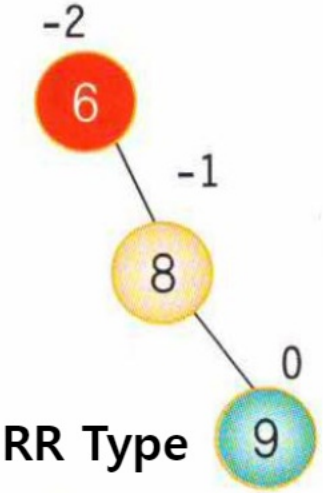
  ② **RR Type**: N이 A의 오른쪽 서브 트리의 오른쪽 서브 트리에 삽입되는 경우

  ③ **LR Type**: N이 A의 왼쪽 서브 트리의 오른쪽 서브 트리에 삽입되는 경우

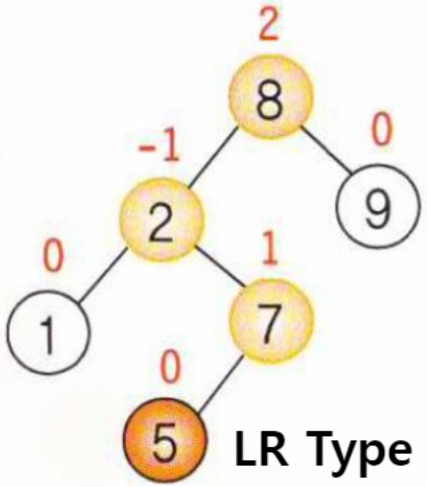
  ④ **RL Type**: N이 A의 오른쪽 서브 트리의 왼쪽 서브 트리에 삽입되는 경우



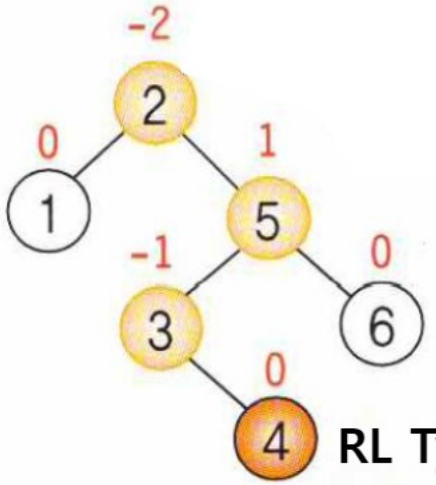
LL Type



RR Type



LR Type



RL Type

서브 트리의 회전 연산

- **회전 연산이란?**
  - ➢ 높이 불균형이 일어난 서브 트리에 대하여 부모 노드와 자식 노드(서브 트리)를 서로 바꾸어 부모 노드를 기준으로 두 서브 트리의 높이 차이를 ±1 이하로 변경하는 연산

- **AVL 트리에서 균형이 깨지는 경우 사용되는 회전 연산의 종류**

① **LL Type** : LL 회전으로 균형을 회복

② **RR Type** : RR 회전으로 균형을 회복

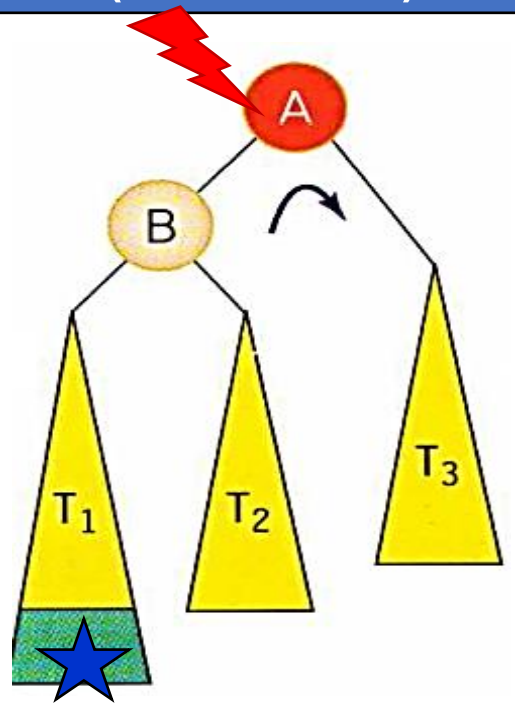
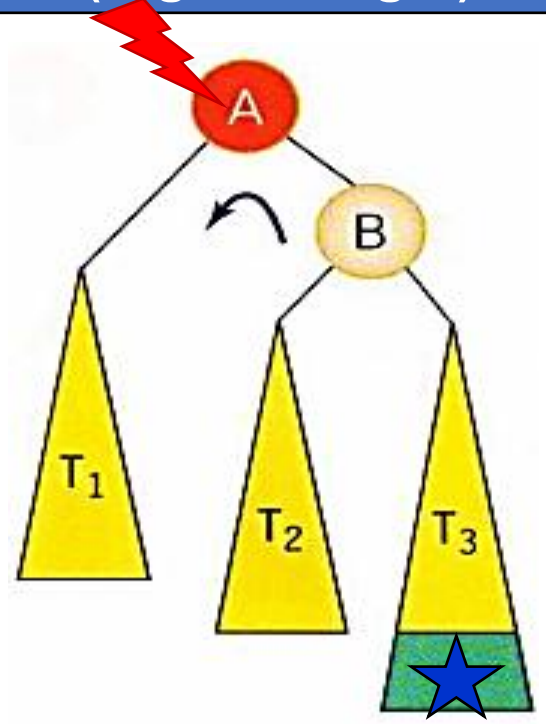
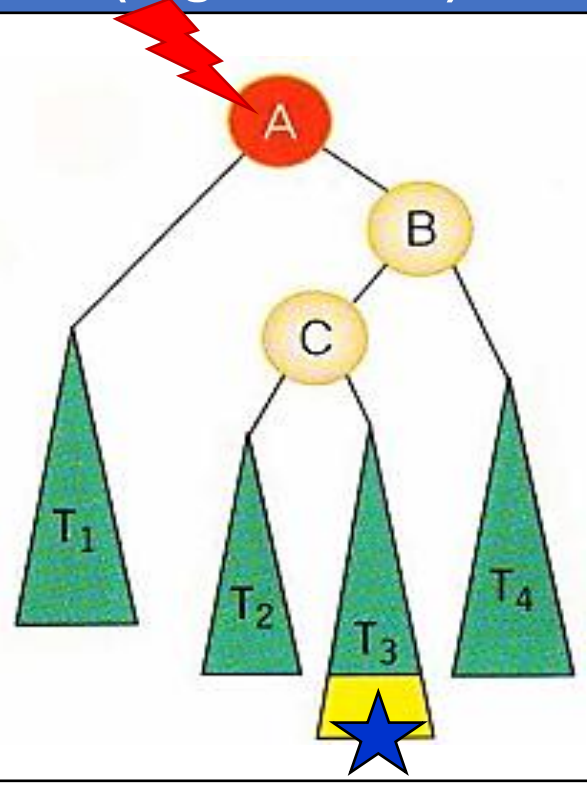
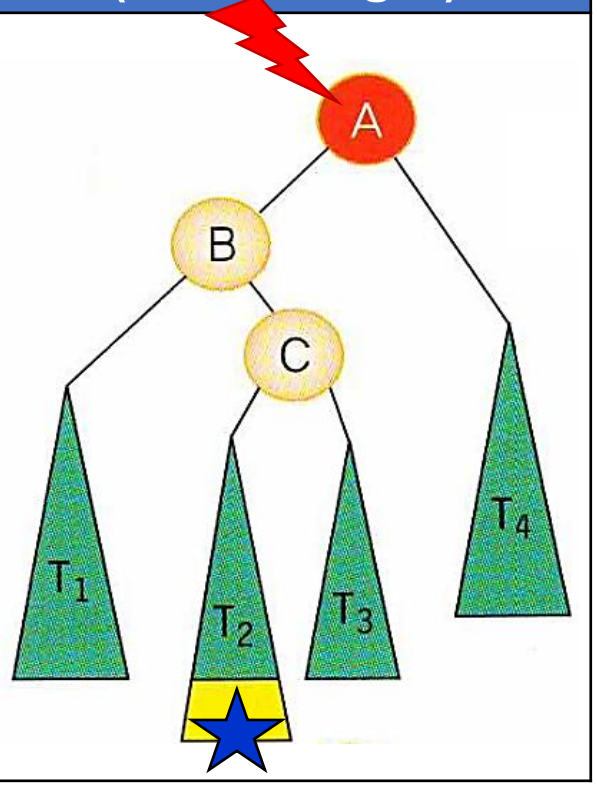
Rotation 연산 1번

③ **LR Type** : RR 회전 후 LL 회전을 통해 균형을 회복

④ **RL Type** : LL 회전 후 RR 회전을 통해 균형을 회복

Rotation 연산 2번

서브 트리의 회전 연산

LL Type (Left → Left)	RR Type (Right → Right)	RL Type (Right → Left)	LR Type (Left → Right)
			

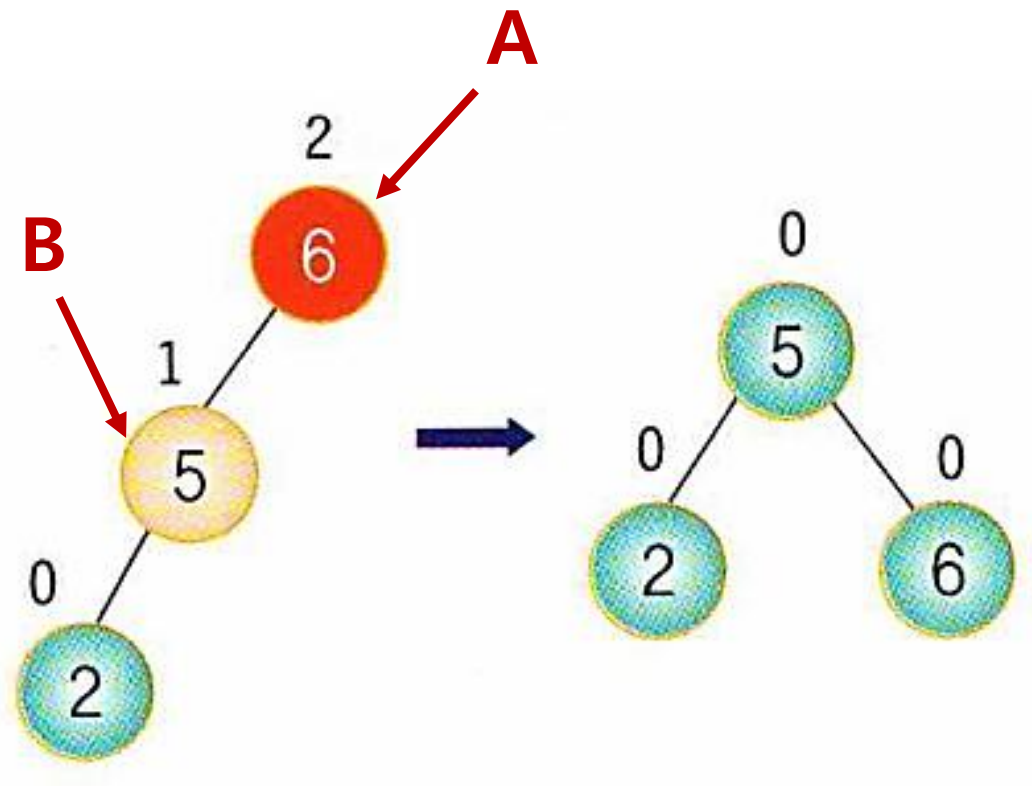
★ 새로운 노드의 삽입 또는 기존 노드의 삭제

두 서브 트리의 높이 불균형이 발생한 노드

서브 트리의 회전 연산 – LL 회전

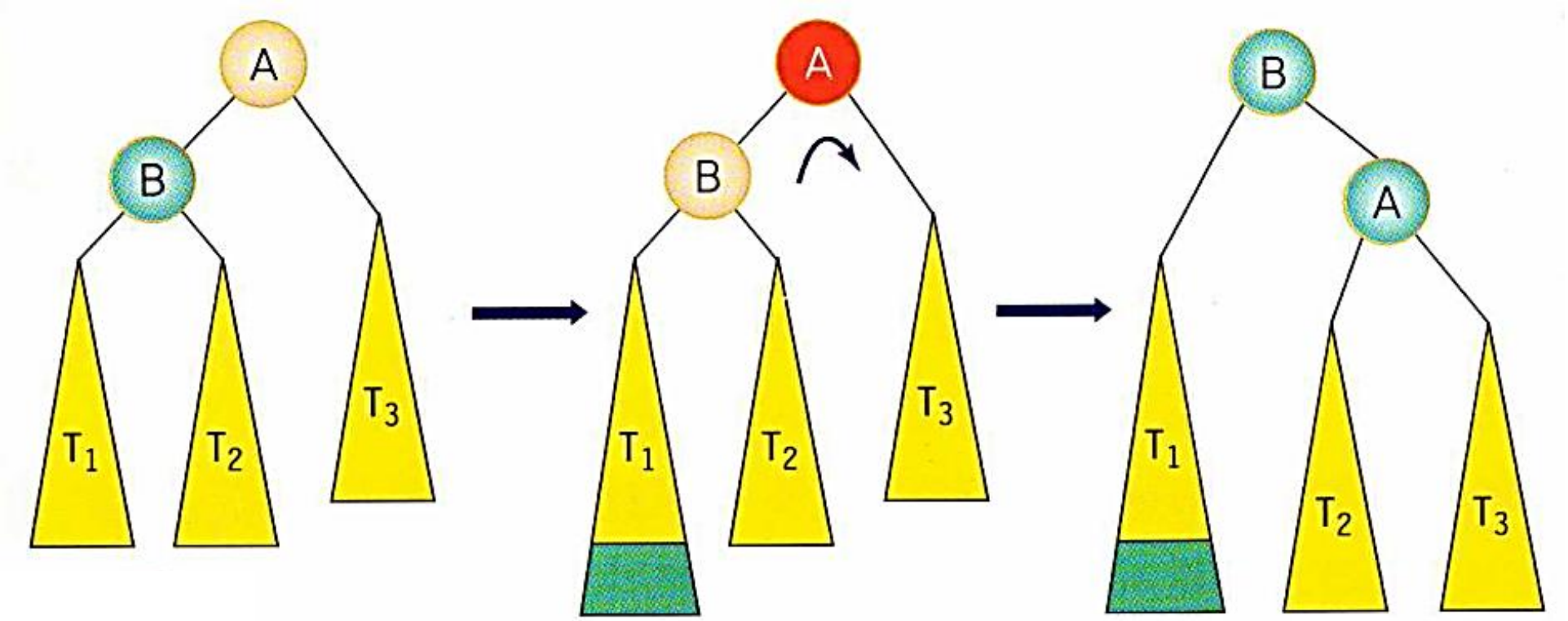
- 불균형이 발생한 노드를 기준으로,
  - ➢ **<u>왼쪽 서브트리의 왼쪽 서브트리</u>**에 특정 노드가 삽입되거나 삭제된 경우

```
Algorithm LL_rotate(A):
    B ← A의 왼쪽 자식;
    B의 오른쪽 자식을 A의 왼쪽 자식으로 만든다;
    A를 B의 오른쪽 자식으로 만든다;
```



서브 트리의 회전 연산 – LL 회전

① B ← A의 왼쪽 자식
② B의 오른쪽 자식을 A의 왼쪽 자식으로 만든다.
③ A를 B의 오른쪽 자식으로 만든다.



T_1에 새로운 노드 삽입 후 노드 A에서 서브 트리 높이의 불균형 발생

T_1의 부모 노드 B를 상위 노드로 이동시키고, 노드 A를 B의 오른쪽 서브 트리로 이동

서브 트리의 회전 연산 – RR 회전

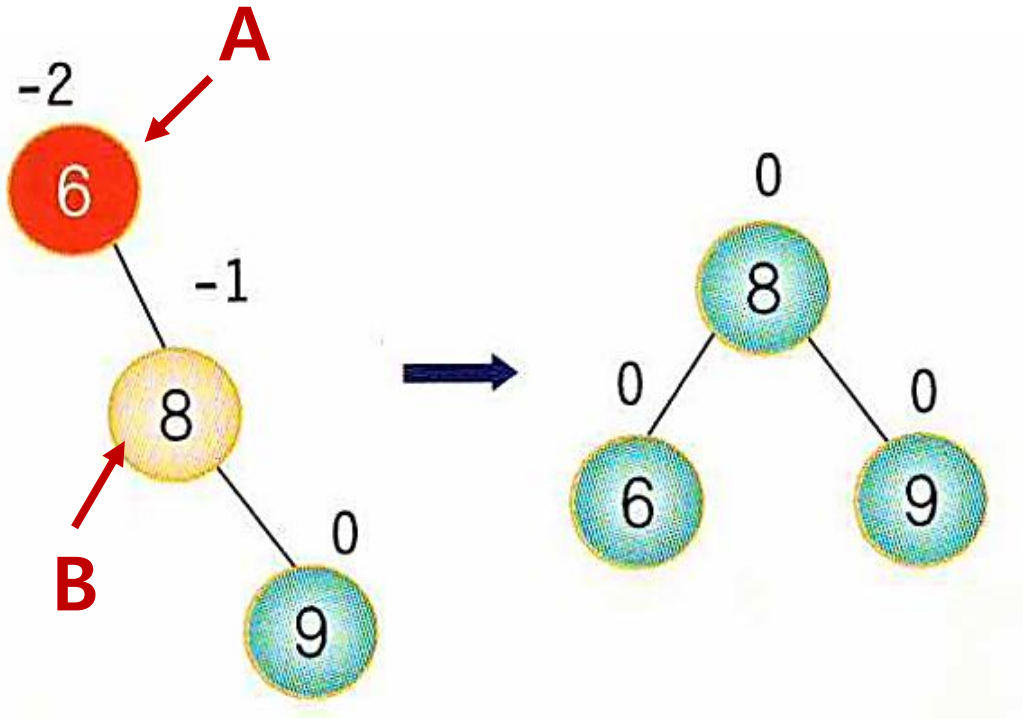
- 불균형이 발생한 노드를 기준으로,
  - ➢ **<u>오른쪽 서브트리의 오른쪽 서브트리</u>**에 특정 노드가 삽입되거나 삭제된 경우

Algorithm RR_rotate(A)

B ← A의 오른쪽 자식;

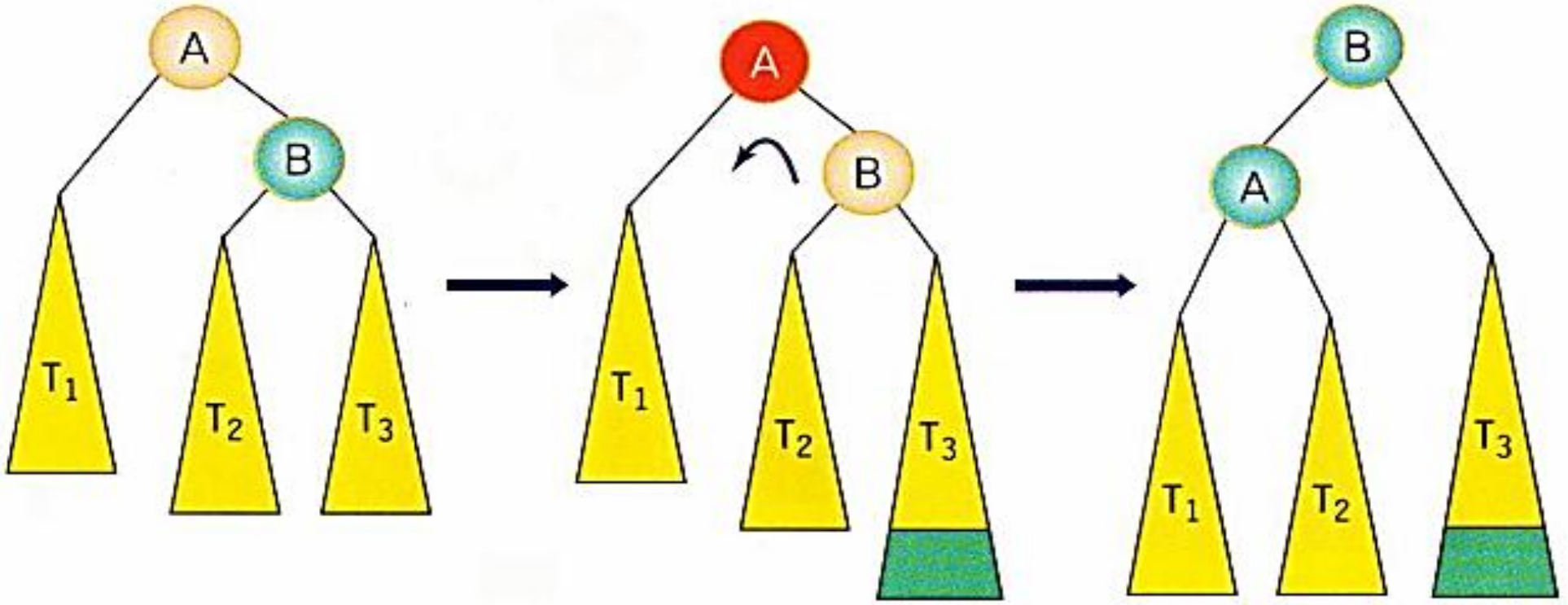
B의 왼쪽 자식을 A의 오른쪽 자식으로 만든다;

A를 B의 왼쪽 자식 노드로 만든다;



서브 트리의 회전 연산 – RR 회전

① B ← A의 오른쪽 자식
② B의 왼쪽 자식을 A의 오른쪽 자식으로 만든다.
③ A를 B의 왼쪽 자식 노드로 만든다.



T_3에 새로운 노드 삽입 후 노드 A에서 서브 트리 높이의 불균형 발생

T_3의 부모 노드 B를 상위 노드로 이동시키고, 노드 A를 B의 왼쪽 서브 트리로 이동

서브 트리의 회전 연산 – RL 회전

LL 회전 + RR 회전

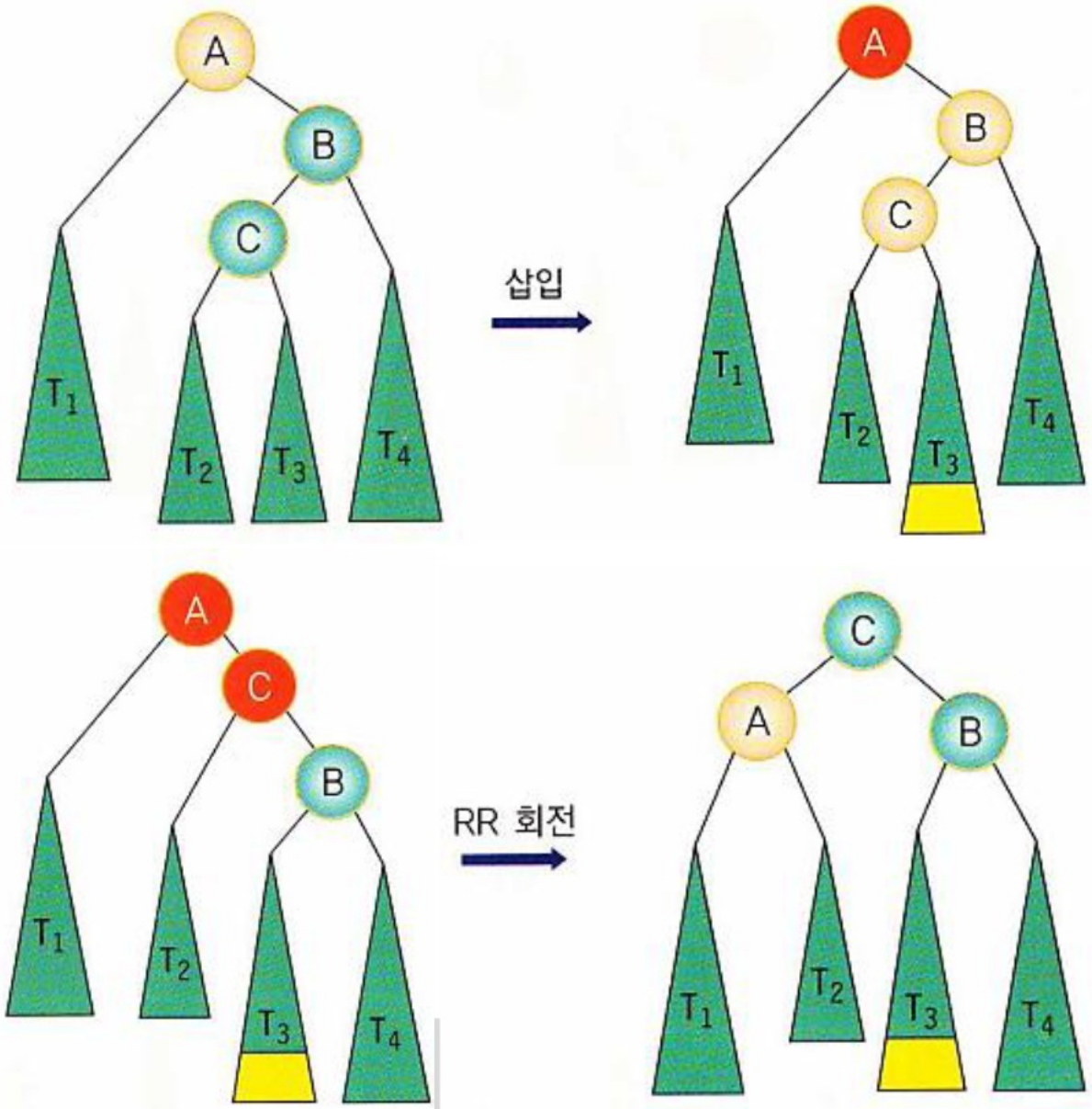
- 불균형이 발생한 노드를 기준으로,
  - ➢ **<u>오른쪽 서브트리의 왼쪽 서브트리</u>**에 특정 노드가 삽입되거나 삭제된 경우

Algorithm RL_rotate(A)

B ← A의 오른쪽 자식;

LL_rotate(B)가 반환하는 노드를 A의 오른쪽 자식으로 만든다;

RR_rotate(A);



서브 트리의 회전 연산 – LR 회전

RR 회전 + LL 회전

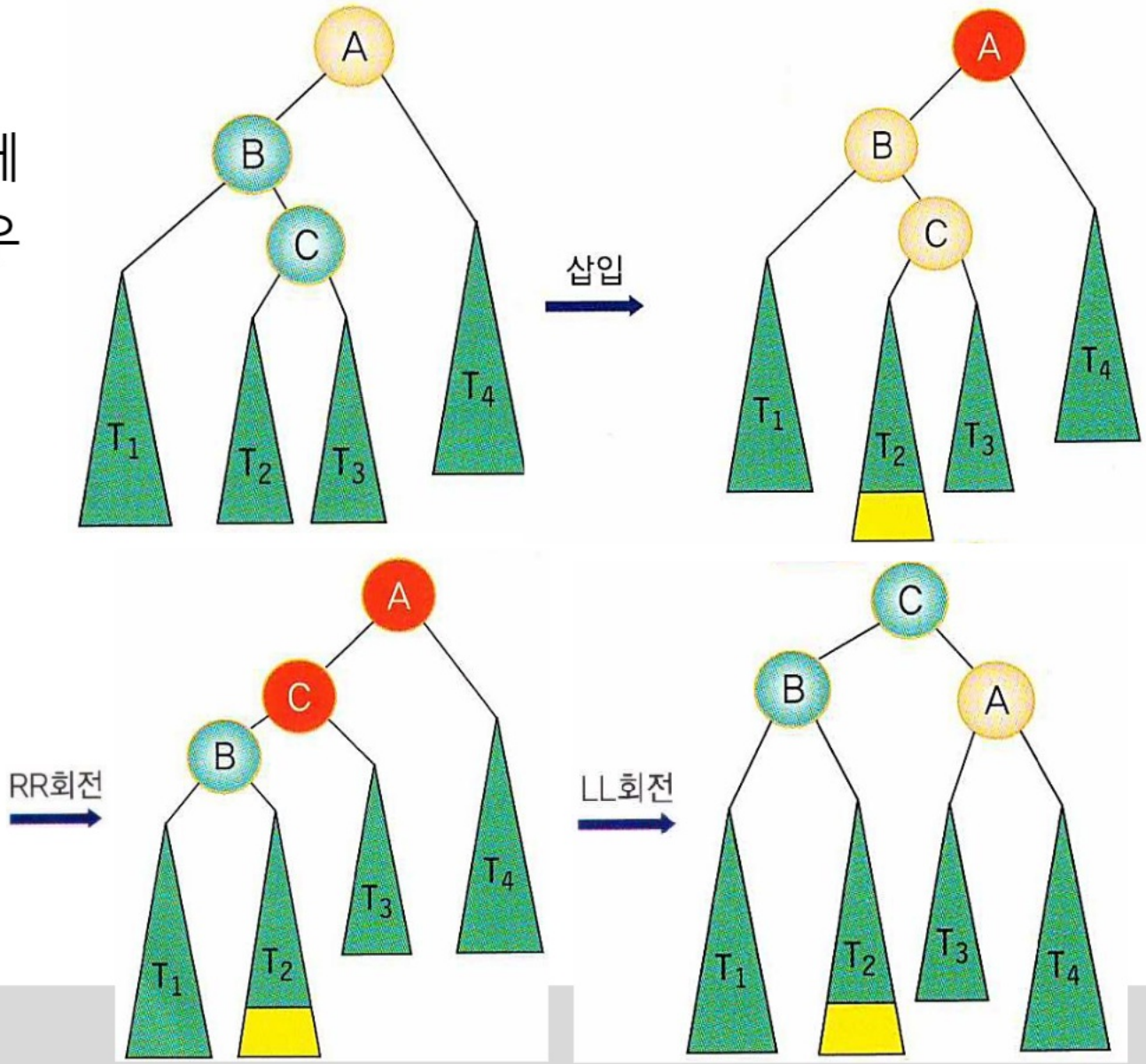
- 불균형이 발생한 노드를 기준으로,
  - ➢ **<u>왼쪽 서브트리의 오른쪽 서브트리</u>**에 특정 노드가 삽입되거나 삭제된 경우

Algorithm LR_rotate(A)

B ← A의 왼쪽 자식;

RR_rotate(B)가 반환하는 노드를 A의 왼쪽 자식으로 만든다;

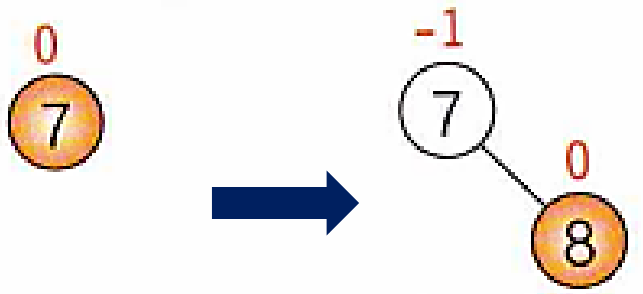
LL_rotate(A);



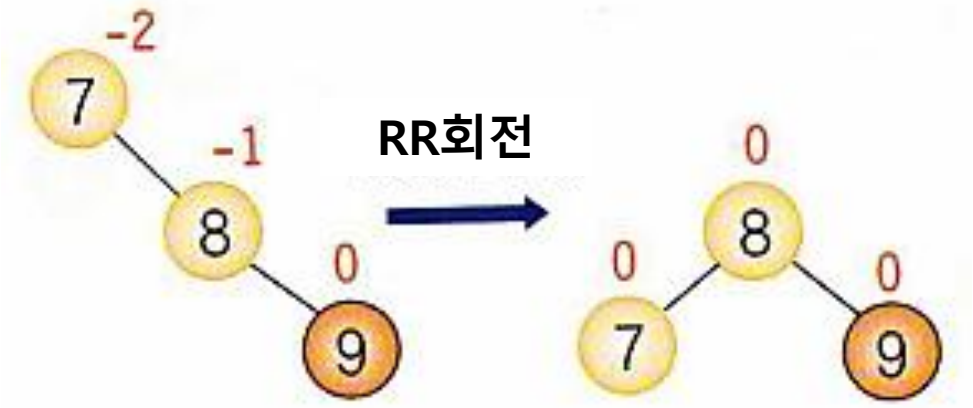
AVL 트리 구축 예제 (1)

- (a) 이진 탐색 트리에 “7” 삽입
- (b) 이진 탐색 트리에 “8” 삽입
- (c) 이진 탐색 트리에 “9” 삽입
  - ✓ 루트 노드 “7”의 균형 인수가 -2

    → 두 서브 트리의 높이 불균형 발생
  - ✓루트 노드 기준 오른쪽 → 오른쪽에 노드가 삽입되어 불균형이 발생하였으므로 “RR 회전” 수행



(a) “7” 삽입 (b) “8” 삽입

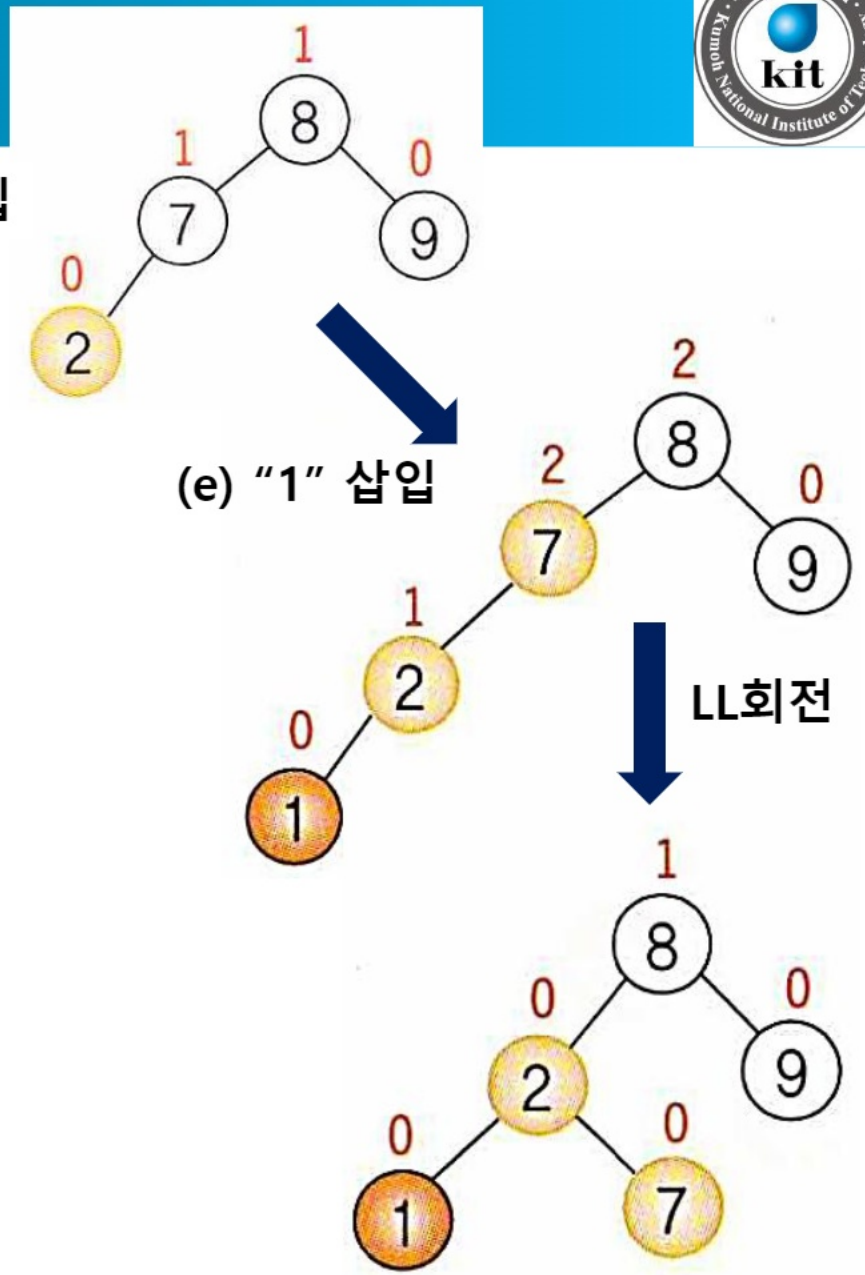


(c) “9” 삽입

AVL 트리 구축 예제 (2)

- (d) 이진 탐색 트리에 “2” 삽입
- (e) 이진 탐색 트리에 “1” 삽입
  - ✓ 루트 노드 “7”의 균형 인수가 2
    → 두 서브 트리의 높이 불균형 발생
  - ✓루트 노드 “7” 기준 왼쪽 → 왼쪽에 노드가 삽입되어 불균형이 발생하였으므로 “LL 회전” 수행

균형 인수가 ±2인 노드 발생 시
Bottom-up으로 노드부터 실행한다
->밑 노드부터 회전 실시

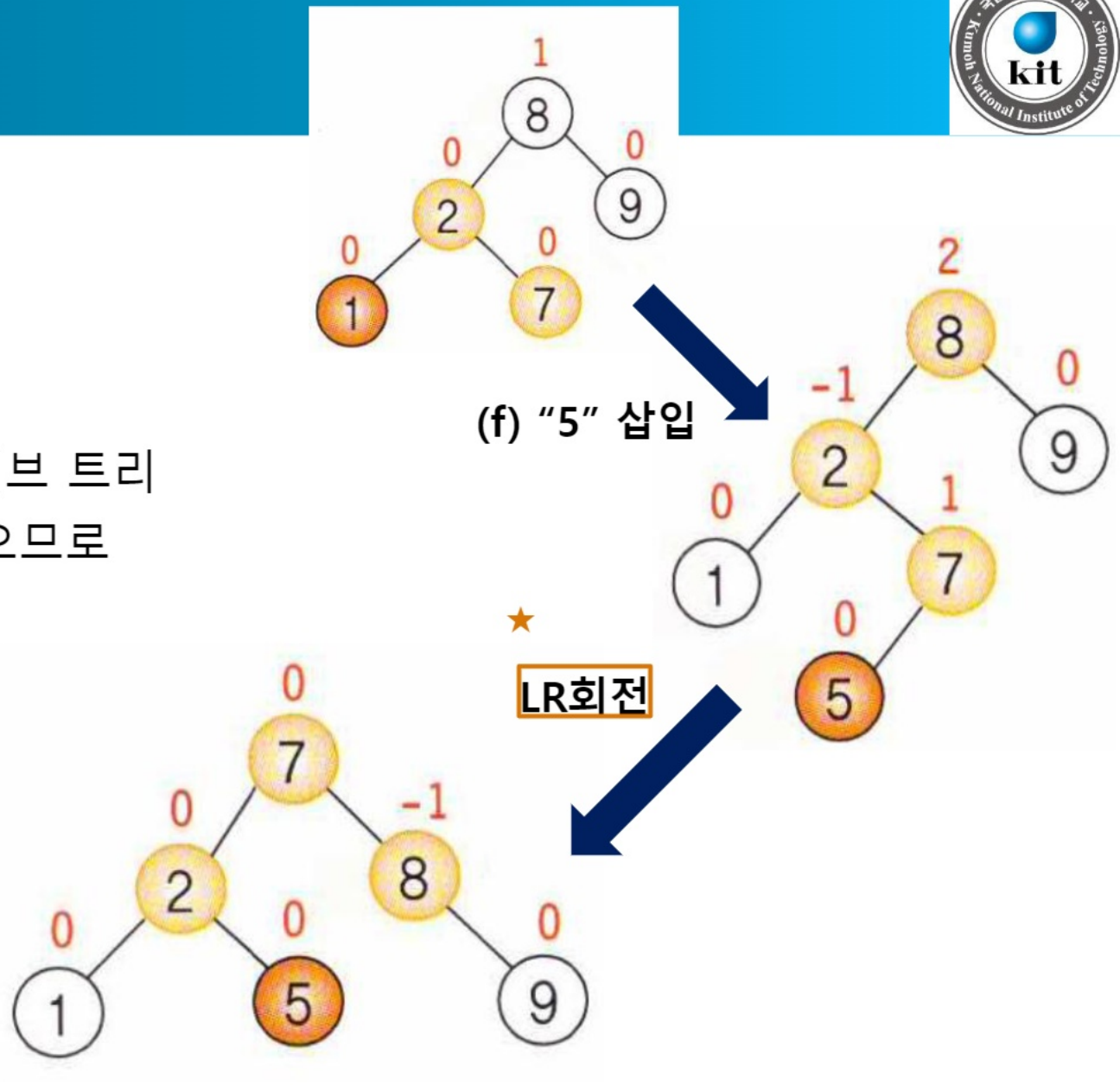


AVL 트리 구축 예제 (3)

- **(f) 이진 탐색 트리에 "5" 삽입**
  - 루트 노드 "8" 의 균형 인수가 2

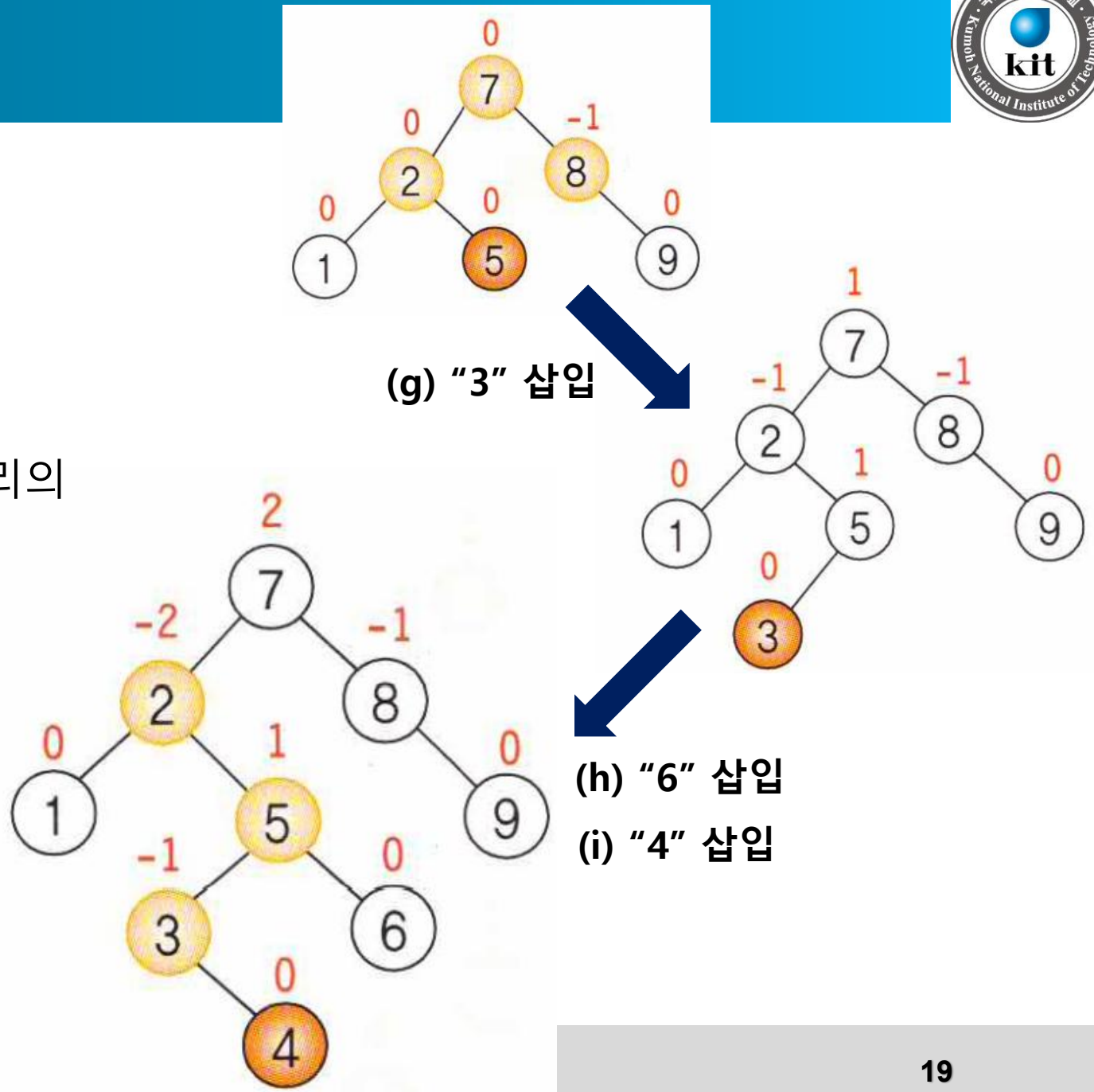
    → 두 서브 트리의 높이 불균형 발생

  ✓ 루트 노드 "8" 기준 왼쪽 → 오른쪽에 서브 트리에 노드가 삽입되어 불균형이 발생하였으므로 "LR 회전" 수행



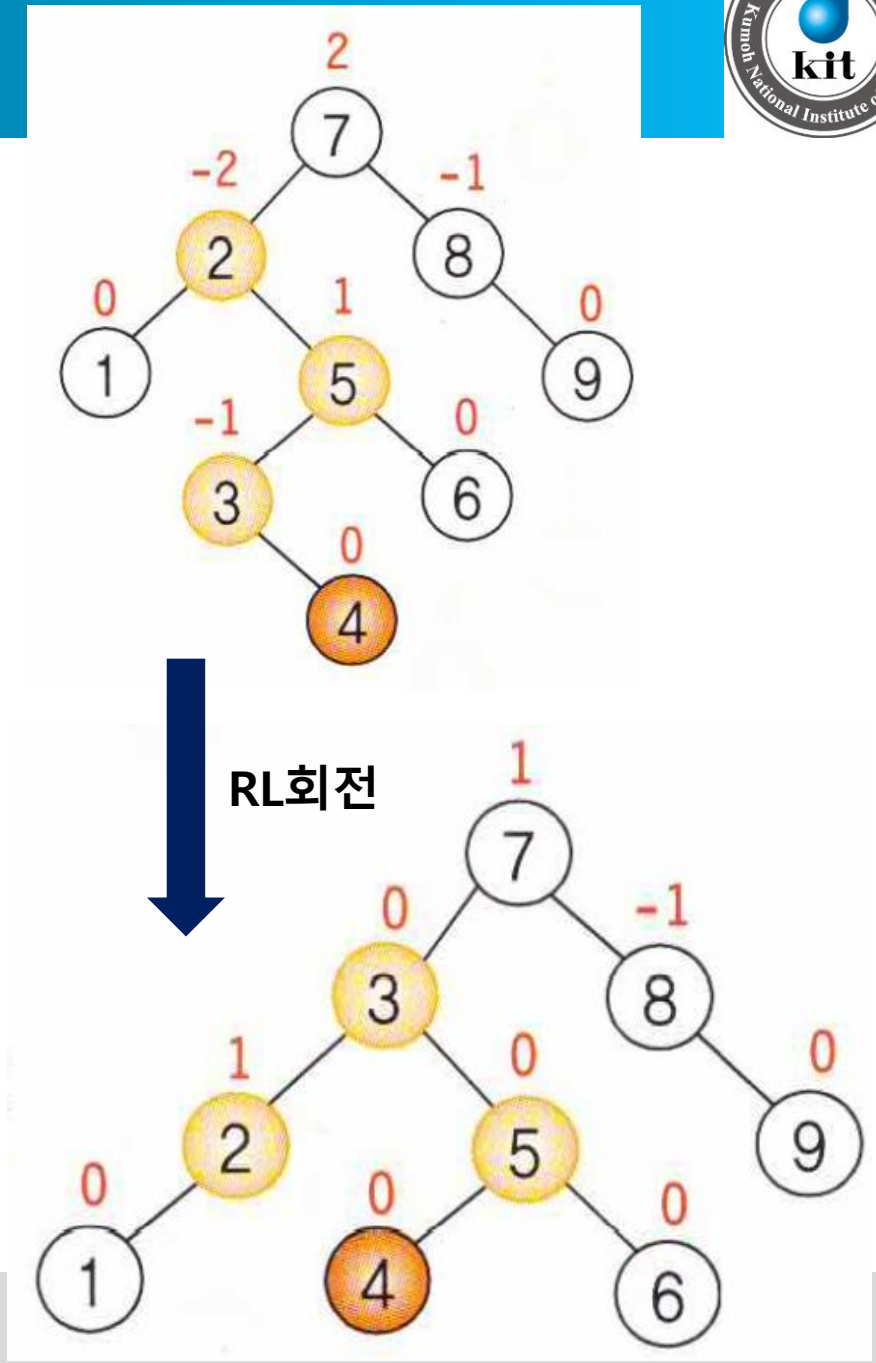
AVL 트리 구축 예제 (4)

- (g) 이진 탐색 트리에 “3” 삽입
- (h) 이진 탐색 트리에 “6” 삽입
- (i) 이진 탐색 트리에 “4” 삽입
  - ✓ 루트 노드 “2” 기준으로 두 서브 트리의 높이 불균형 발생(균형 인수가 -2)



AVL 트리 구축 예제 (5)

- **(i) 이진 탐색 트리에 “4” 삽입**
  - ✓ 루트 노드 “2” 기준으로 오른쪽 → 왼쪽 서브트리에 노드 “4”가 추가되어 불균형이 발생 (노드 “2”의 균형 인수가 -2)
  - ✓ “RL 회전” 수행



AVL 트리 이해를 위한 유용한 자료

- **AVL 트리 C 소스코드**
  - LMS에 업로드되어 있는 “AVLTree.c” 파일 참고
- **AVL 트리 삽입/삭제 예제 animation**
  - **Animation I.** https://www.youtube.com/watch?v=mGF61O21W-c
  - **Animation II.** https://www.youtube.com/watch?v=up1FY5l8CEk

AVL 트리의 복잡도 분석

노드 n개를 포함하는 균형 이진 탐색 트리의 높이는 $h_n = \log_2 n$이다.

- **데이터의 검색**
  - AVL 트리에서 데이터를 검색하는 방법은 일반 이진 트리와 동일함.
  - 즉, 루트 노드에서 시작하여, top-down 방식으로 데이터를 찾을 때 까지 각 레벨별로 이동하면서 탐색을 진행함.
  - 이 때 발생할 수 있는 최악의 경우는 현재 찾고자 하는 데이터가 트리 내에 존재하지 않거나, 해당 데이터가 leaf level에 존재하는 경우임.
    - ➔ 즉, root에서 leaf level까지 검색이 진행되는 경우가 최악의 경우
  - 따라서 데이터 검색에 소요되는 최악의 경우의 시간 복잡도는 $O(\log n)$이다.

AVL 트리의 복잡도 분석

노드 n개를 포함하는 균형 이진 탐색 트리의 높이는 $h_n = \log_2 n$이다.

- **데이터의 삽입 또는 삭제**
  - ➢ 새로운 노드가 삽입될 위치 또는 삭제할 노드의 위치를 탐색: $O(\log n)$
  - ➢ 서브트리 높이의 불균형 발생 시 불균형이 발생한 노드의 서브 트리를 재 구조화(즉, 회전 연산)하는 데 소요되는 시간 복잡도: $O(c)$
  - ➢ 따라서 데이터의 순수 삽입/삭제에 소요되는 비용과 트리 재구성에 소요되는 시간 복잡도를 모두 더하면 $O(\log n + c) = O(\log n)$

요약

- 트리의 높이가 항상 $\log n$에 가깝게 유지될 수 있도록, 서브 트리 간의 높이 균형을 자동으로 조정하는 이진 탐색 트리를 "균형 이진 탐색 트리"라고 한다.
- 두 서브 트리의 높이 차이가 ±1 이하인 경우 "높이 균형을 이룬다"고 한다.
- 임의의 이진 탐색 트리 T가 높이 균형일 때 T를 "**AVL 트리**"라고 한다. → 그림을 그리면서 연습할 것
- AVL 트리에서 균형이 깨지는 경우는 트리 내 높이 균형이 깨지는 위치에 따라서 LL, RR, RL, LR 타입으로 구분할 수 있다.
- 회전 연산이란 높이 불균형이 일어난 노드의 서브 트리에 대하여 부모 노드와 자식 노드(서브 트리)를 서로 바꾸어 부모 노드를 기준으로 두 서브 트리의 높이 차이를 ±1 이하로 변경하는 연산을 의미한다.
- AVL 트리의 삽입, 삭제, 탐색에 소요되는 최악의 경우의 시간 복잡도는 $O(\log n)$이다.

그 외에도 Red-black 트리, Splay 트리 등등이 있음

ex. 삽입, 삭제의 수행시간은?
A: 둘 다 O(log n)

Q & A